

PARKING LOT MANAGEMENT SYSTEM

System Design and Implementation Document

React | Node.js | Express | PostgreSQL | MongoDB | Redis

1. Project Overview

The Parking Lot Management System manages a fixed lot with 5 bike bays, 5 car bays, and 2 truck bays. It provides authenticated operator workflows for vehicle entry, ticket generation, vehicle exit, fare calculation, live capacity, active vehicles, and parking history.

2. Objectives

- Allocate one compatible bay per vehicle and generate a unique ticket.
- Keep PostgreSQL as the source of truth for transactional parking state.
- Prevent duplicate active vehicles and double assignment of a bay.
- Demonstrate correct behavior when simultaneous requests compete for the last bay.
- Provide a responsive operator dashboard with auditable activity.

3. Functional Requirements

Capacity: B1-B5 for bikes, C1-C5 for cars, and T1-T2 for trucks. Parking accepts a vehicle number and type, checks availability, assigns one bay, and creates a ticket. Exit accepts a ticket ID or vehicle number, calculates duration and fare, completes the record, and releases the bay. Pricing is Rs 30 up to 180 minutes, Rs 85 from 181-360 minutes, and Rs 120 above 360 minutes.

4. System Architecture

Architecture flow: React browser dashboard -> Express REST API -> JWT middleware -> PostgreSQL for transactional state. Express also uses Redis for availability caching and MongoDB for activity logs. React sends JSON requests to Express. Express validates input and applies business rules.

5. Technology Stack

Frontend: React with Vite. Backend: Node.js and Express. Primary database: PostgreSQL for slots, users, tickets, records, constraints, and transactions. Secondary database: MongoDB for unstructured activity and audit logs. Cache: Redis for availability counts with invalidation after parking and exit.

6. Frontend Architecture

The React client provides a login screen and authenticated control-room dashboard with capacity cards, parking and exit forms, latest transaction results, active vehicles, recent history, and sign-out. The layout is responsive for desktop, tablet, and mobile screens.

7. Backend Architecture

Express exposes authentication, parking, exit, availability, active-record, history, and health endpoints. Services separate fare calculation, activity logging, cache refresh, and transactional parking operations. Centralized error handling returns consistent JSON without exposing internal database details.

8. PostgreSQL Database Design

parking_slots stores unique slot number, vehicle type, status, and timestamps. parking_records stores unique ticket, vehicle number/type, slot foreign key, entry/exit times, duration, fare, and ACTIVE/COMPLETED status. app_users stores unique username and bcrypt password hash. Partial unique indexes prevent more than one active record for a vehicle or slot.

9. Redis Caching

Redis stores availability objects under parking:slots:BIKE, parking:slots:CAR, and parking:slots:TRUCK. Availability reads use a short-lived read-through cache. The cache is refreshed on a miss and invalidated/refreshed after successful parking and exit. PostgreSQL remains authoritative.

10. MongoDB Activity Logging

MongoDB stores activity_logs documents containing event type, timestamp, vehicle number, ticket ID when available, message, and metadata. Logging is best effort and does not roll back a successful PostgreSQL transaction.

11. Authentication and Security

- JWT bearer tokens protect parking endpoints; login remains public.
- Passwords are stored as bcrypt hashes.
- Helmet, CORS, JSON limits, validation, and write rate limiting are enabled.
- SQL values are parameterized and environment variables hold configuration.
- Client errors do not expose SQL statements, stack traces, or credentials.

12. Parking Allocation Logic

1. Begin a PostgreSQL transaction. 2. Normalize and validate the vehicle. 3. Check for an active vehicle. 4. Select one compatible AVAILABLE slot with FOR UPDATE SKIP LOCKED. 5. If none exists, roll back and return Parking Full. 6. Mark the slot OCCUPIED and insert the active ticket. 7. Commit, then refresh the cache and write the activity event.

13. Ticket Generation

Each successful parking request generates a ticket using a timestamp and random UUID fragment. The ticket ID is unique in PostgreSQL and is returned with vehicle, type, bay, and entry time.

14. Exit and Fare Calculation

An exit locates an active record using ticket ID or normalized vehicle number. The record is locked inside a transaction. The backend calculates rounded-up whole-minute duration, applies the fare service, marks the record COMPLETED, records exit time, and marks the bay AVAILABLE. Fare bands are Rs 30 up to 180 minutes, Rs 85 from 181-360 minutes, and Rs 120 above 360 minutes.

15. Concurrency Handling

When two requests compete for the final compatible bay, the first transaction locks and updates the available slot. The second request skips the locked row and finds no available slot, so it receives Parking Full. Partial unique indexes provide additional protection. Redis is not the concurrency authority.

16. API Endpoints

POST /api/auth/login (public): returns JWT. POST /api/parking/park (JWT): parks a vehicle. POST /api/parking/exit (JWT): exits by ticket or vehicle number. GET /api/parking/slots (JWT): availability. GET /api/parking/active?page=1&limit=25 (JWT): active records. GET /api/parking/history?page=1&limit=25 (JWT): completed records. GET /api/health (public): dependency health.

17. Error Handling

Errors use { success: false, message, errorCode }. Validation errors return 400, authentication failures 401, missing active records 404, and conflicts such as duplicate active vehicles or full categories 409. Unexpected errors return a generic 500 response.

18. Testing and Test Results

The suite covers valid parking, invalid vehicle number/type, full capacity, duplicate active vehicles, exit by ticket, exit by vehicle number, invalid/repeated exits, fare rules, slot release, and simultaneous competition for the final car slot. Verified results: 10 backend tests passed; concurrency passed; frontend build passed; frontend lint passed; Docker Compose validation passed; PostgreSQL, MongoDB, and Redis health checks passed; authenticated API smoke test passed.

19. Docker and Deployment

Docker Compose defines PostgreSQL 16, MongoDB 7, and Redis 7. PostgreSQL mounts schema and seed files for a new volume. Existing databases created before authentication need database/migration-auth.sql once. Copy server/.env.example to server/.env, start the API in server, and start the client in client.

20. Project Structure

client/ contains the Vite React application. server/src/ contains the Express app, authentication, configuration, routes, and services. server/tests/ contains API, fare, and concurrency tests. database/ contains schema, seed, and migration files. docker-compose.yml defines services. README.md contains setup and testing instructions.

21. Assumptions and Limitations

- Fixed capacity and vehicle-type-compatible bays.
- UTC timestamps and rounded-up whole-minute billing.
- One operator role and no vehicle-owner workflow.
- Best-effort MongoDB logging outside the PostgreSQL transaction.
- Development seed account must be replaced before production.
- Future scaling could use stateless API instances, read replicas, pooling, and a durable audit queue.

22. Submission and Security

Do not submit server/.env or files containing real connection strings, JWT secrets, API keys, tokens, passwords, or private credentials. server/.env.example is safe because it contains placeholders. Do not submit node_modules, client/dist, coverage output, database dumps, or personal data. The development account and bcrypt seed hash are demo-only values and must not be reused in production.

